2. Final Project Report and Presentation Requirements The final project presentation and report evaluate technical implementation, experimentation, analysis, and understanding. The presentation should summarize the report; the report must include full technical detail.

Required Sections

1. Project Goal Restate the project objective. • Briefly reflect on whether the goal was achieved.

Our project Goal is to predict credit card fraudulent transactions to enable early detection and fraud prevention. Yes, our goal is achieved, our model predicts fraud transactions with the 99% accuracy with loss 0.01%.

2. Input and Ground Truth:

Describe the final input representation and ground truth labels. • Note any changes from the proposal stage.

Input :

- Elapsed time between the transactions in seconds
- 28 PCA transformed features (V1 to V28)
- Transaction Amount

Ground Truth labels: binary classification (Class =1 is fradualent, class=0 is not fradualent)

3. Dataset Description • Describe the dataset(s) used. • Include dataset size, class distribution, and source.

We used the **Credit Card Fraud Detection** Dataset from Kaggle.

This data set size is 284,807 records, two classes- fraud-1 (492), not fraud -0 (284,315)

Data is in Comma delimited format.

The dataset is public; however, the original feature names are not provided due to confidentiality.

4. Data Processing • Describe preprocessing steps (e.g., normalization, resizing). • Clearly define training, validation, and test splits.

- Normalization was performed using StandardScaler.
- Each feature was transformed to have:
    - Mean = 0
    - Standard deviation = 1

Only the training data was used to fit the scaler. Validation and test sets were transformed using the same fitted scaler to prevent data leakage.

Because the dataset is highly imbalanced, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training set only.

SMOTE generates synthetic minority class samples using nearest neighbors, which helps the CNN learn fraud patterns more effectively.

Since a 1D Convolutional Neural Network was used each sample originally had shape: (30,). It was reshaped to (1, 30). This adds a channel dimension required by Conv1D layers

The dataset was split into three subsets using stratified sampling to preserve the class distribution.

Split	**Percentage**	**Purpose**
Training	70%	Model learning
Validation	15%	Hyperparameter tuning & monitoring
Test	15%	Final model evaluation

• Explain how data leakage was avoided.

Using SMOTE technique, data leakage was avoided.

5. Methodology

5.1 Baseline Model •

Describe the baseline model in detail. • Report baseline performance.

Baseline model is logistic regression model which is a simple linear classifier.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\savit\Desktop\Deeplearning> & C:/Users/savit/AppData/Local/Programs/Python/Python313/python.exe c:/Users
/savit/Desktop/Deeplearning/CNN_model_with_and_withoutsmote_withlr0.01_epochs15.py
ROC-AUC  : 0.9479209442982657

Metrics saved to results_cnn_smote_metrics.csv
PS C:\Users\savit\Desktop\Deeplearning> & C:/Users/savit/AppData/Local/Programs/Python/Python313/python.exe c:/Users
/savit/Desktop/Deeplearning/baseline.py
Dataset description: (284807, 31)
Train data size: 199364
Validation datasize: 42721
Test data size: 42722

Training Logistic Regression ...

 Logistic Regression metrics
Accuracy : 0.9991105285333084
Precision: 0.8913043478260869
Recall   : 0.5540540540540541
F1 Score : 0.6833333333333333
ROC-AUC  : 0.9776210157822426
PS C:\Users\savit\Desktop\Deeplearning>
```

5.2 CNN Model Architecture • Describe the CNN architecture used. • Explain key design choices (e.g., number of layers, kernel sizes).

The Convolutional Neural Network (CNN) used in this project is a 1D CNN architecture designed for tabular transaction data from the credit card fraud dataset.

Input Layer

- Input size: 30 numerical features
- Reshaped to: (batch_size, 1, 30)
- The extra dimension allows use of Conv1D

Layer 1: Convolutional Layer

Input channels: 1

Output channels: 32

Kernel size: 3

Activation: ReLU

Layer 2: Convolutional Layer

Input channels: 32

Output channels: 64

Kernel size: 3

Activation: ReLU

Max Pooling Layer:

Pool size=2

Kernel=2

Reduces feature dimension by half

Keeps strongest activations

Reduces overfitting

Decreased computation

Makes model more robust

Fully Connected Layer 1:

Flattens convolution output

64 neurons

Activation: ReLU

Dropout applied (0.2)

Output Layer: Binary classification (Fraud vs Non-Fraud)

Sigmoid applied during evaluation

5.3 Training Details and Reproducibility • Specify the deep learning framework used. • Describe training settings (optimizer, learning rate, batch size, epochs). • Report random seeds and hardware used.

The model was implemented using:

- Framework: PyTorch
- Supporting libraries:
    - NumPy
    - Scikit-learn (preprocessing & metrics)

- Imbalanced-learn (SMOTE)
- Matplotlib (visualization)

PyTorch was chosen due to:

- Flexibility in defining custom neural networks
- Explicit control over training loops

Optimizer= Adam, learning_rates=0.01, batch size=256, epochs=30

Random seed=123 for reproducibility , CPU-hardware and SMOTE was used for class imbalance handling.

6. Performance Improvement Techniques You must implement and analyze at least three techniques (such as Data augmentation, Different dropout rates, Batch normalization) For each technique: • Describe how it was applied. • Report and analyze its effect on performance.

I used below three techniques.

1. CNN base model: CNN model with smote, dropout_rate=0.2, learning rate =0.01 and epochs=30
2. CNN model with smote, dropout_rate=0.2, learning rate =0.001 and epochs=30
3. CNN model with smote, dropout_rate=0.2, learning rate =0.01 and epochs=15
4. CNN model without smote CNN model with smote, dropout_rate=0.2, learning rate =0.01 and epochs=30

Please see the model metrics for each model in the below table.

Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC	Training_Loss_Final	SMOTE
CNN base model with learning_rate=0.01 and epochs=30	0.9988765	0.6444444	0.7837838	0.70731707	0.9402946	0.0032	Applied
CNN model with learning_rate=0.001 and epochs=30	0.9992978	0.8055556	0.7837838	0.79452055	0.9627718	0.001	Applied
CNN model with learning_rate=0.01 and epochs=15	0.9988998642385656	0.6483516483516484	0.7972972972972973	0.7151515151515152	0.9479209442982657	0.00461811	Applied
CNN model with learning_rate=0.01 and epochs=30	0.9947567997752914	0.2262773722627737	0.8378378378378378	0.3563218390804598	0.9612688976258196	0.7469	Not Applied

We can see clearly in the above table, the effect on model's performance comparison in three different techniques.

Overall CNN model with learning rate-0.001 and epochs=30 is the beet model out of the 4 models.

7. Performance Evaluation •

Clearly define evaluation metrics. •

1. Accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Measures overall correctness of the model.

Limitation: On highly imbalanced datasets (like fraud detection), accuracy can be misleading because predicting the majority class (normal transactions) dominates the score.

2. Precision (Positive Predictive Value)

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives + False Positives}}$$

Measures how many of the predicted fraud cases are actually fraud.

High precision → model makes fewer false alarms.

Important when false positives are costly (e.g., flagging legitimate transactions).

3. Recall (Sensitivity / True Positive Rate)

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives + False Negatives}}$$

Measures how many actual fraud cases were correctly identified.

High recall → model catches more frauds.

Important when missing a fraud (false negative) is costly.

4. F1 Score

$$\mathrm{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean of precision and recall.

Provides a single score that balances false positives and false negatives.

Useful for imbalanced datasets where both precision and recall matter.

5. ROC-AUC (Receiver Operating Characteristic – Area Under Curve)

Plots True Positive Rate (Recall) vs False Positive Rate at various threshold settings.

AUC (Area Under the Curve) summarizes the model's ability to discriminate between classes:

ROC-AUC
= Probability that a randomly chosen fraud transaction is ranked higher than a normal one

Value range: 0.5 (random) → 1.0

Less sensitive to class imbalance than accuracy but still may overestimate performance when the positive class is extremely rare.

6. PR-AUC (Precision-Recall Area Under Curve)

Plots Precision vs Recall for all thresholds.

PR-AUC summarizes the trade-off between precision and recall:

Particularly informative for highly imbalanced datasets like credit card fraud detection, where the positive class is rare.

A higher PR-AUC indicates better performance in identifying frauds without generating too many false positives.

7. Confusion Matrix

	Predicted Normal	Predicted Fraud
Actual Normal	True Negative (TN)	False Positive (FP)
Actual Fraud	False Negative (FN)	True Positive (TP)

Provides a detailed breakdown of correct and incorrect predictions.

Helps interpret which types of errors are being made.

Present quantitative results using tables and/or plots.

Model	**Accuracy**	**Precision**	**Recall**	**F1_Score**	**ROC_AUC**	**Training_Loss_Final**	**SMOTE**
Logistic regression model	**0.9991053**	**0.891304**	**0.554054**	**0.6833333**	**0.977621**		**N/A**
CNN base model with learning_rate=0.01 and epochs=30	0.9988765	0.6444444	0.7837838	0.70731707	0.9402946	0.0032	Applied
CNN model with learning_rate=0.001 and epochs=30	0.9992978	0.8055556	0.7837838	0.79452055	0.9627718	0.001	Applied
CNN model with learning_rate=0.01 and epochs=15	0.9988998642385656	0.6483516483516484	0.7972972972972973	0.7151515151515152	0.9479209442982657	0.00461811	Applied
CNN model with learning_rate=0.01 and epochs=30	0.9947567997752914	0.22627737226277372	0.8378378378378378	0.3563218390804598	0.9612688976258196	0.7469	Not Applied
Foundation model with learning_rate=0.01 and epochs=31	0.9946866	0.2238267	0.83783783	0.35327635	0.9648944	0.72972743	Applied

8. Comparison of Approaches • Compare the baseline model with improved CNN models. • Compare different architectures or training strategies. • Identify which approach performed best and explain why.

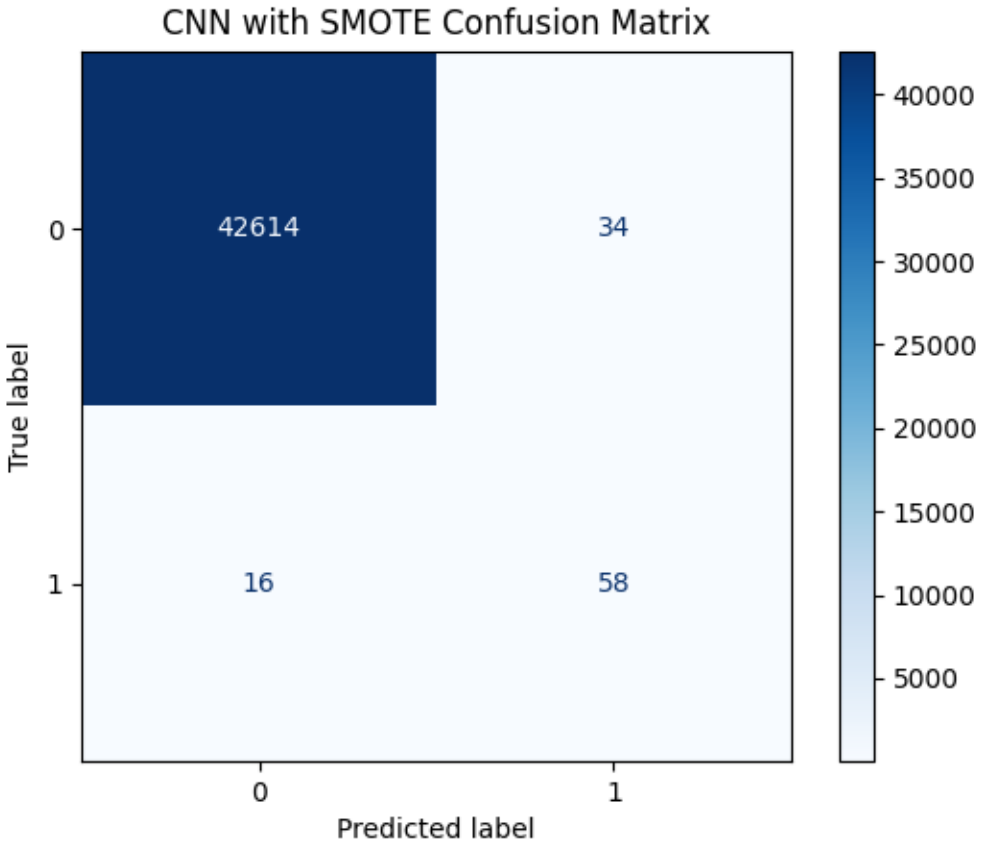
Overall CNN model with learning rate-0.001 and epochs=30 is the beet model out of the 4 models.

9. Foundation Model Comparison • Implement a foundation model approach (e.g., CLIP). • Compare its performance with your CNN-based model. • Analyze performance differences and explain possible reasons.

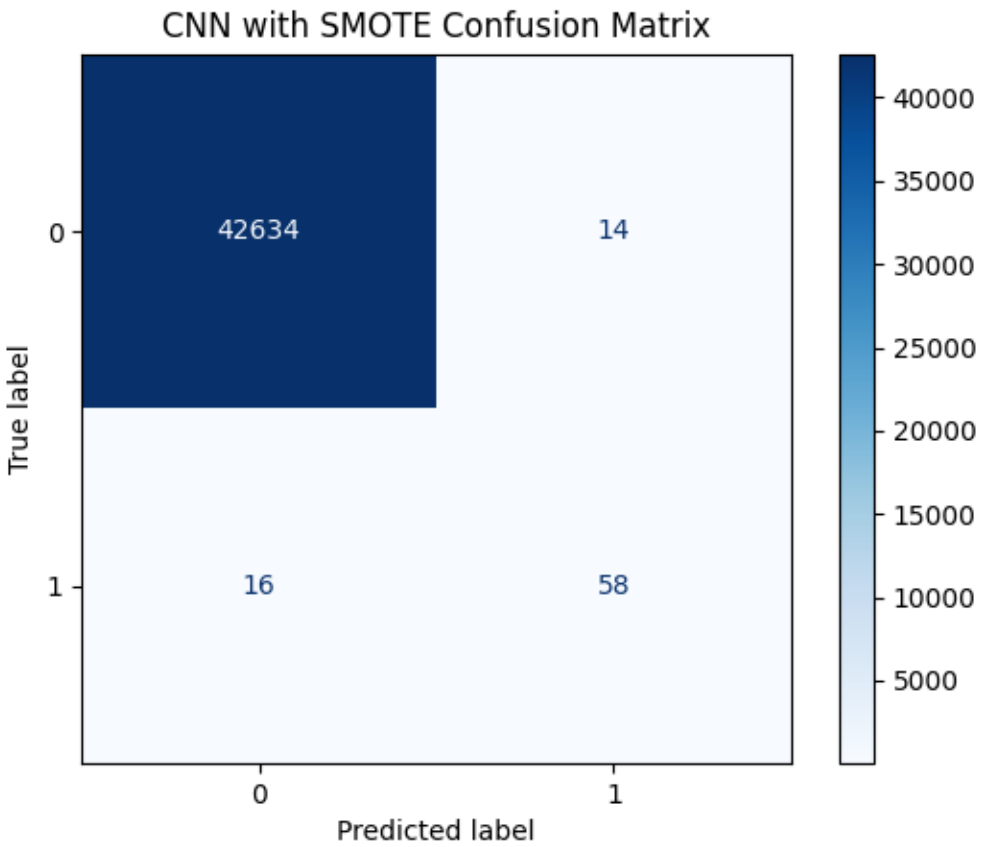
Model	**Accuracy**	**Precision**	**Recall**	**F1_Score**	**ROC_AUC**	**Training_Loss_Final**	**SMOTE**
CNN base model with learning_rate= 0.01 and epochs=30	0.998876	0.644444	0.783784	0.707317	0.940295	0.0032	Applied
Foundation model with learning_rate= 0.01 and epochs=31	0.994687	0.223827	0.8378378378378378	0.353276	0.964894	0.729727	Applied

10. Error and Failure Analysis • Analyze common failure cases. • Provide qualitative or quantitative examples of errors. • Discuss patterns observed in misclassifications.

1. CNN_confusionmatrix_withSmote_withlr0.001_andepochs15_dropoutrate0.2



2. CNN_confusionmatrix_withSmote_withlr0.001_andepochs30_dropoutrate0.2

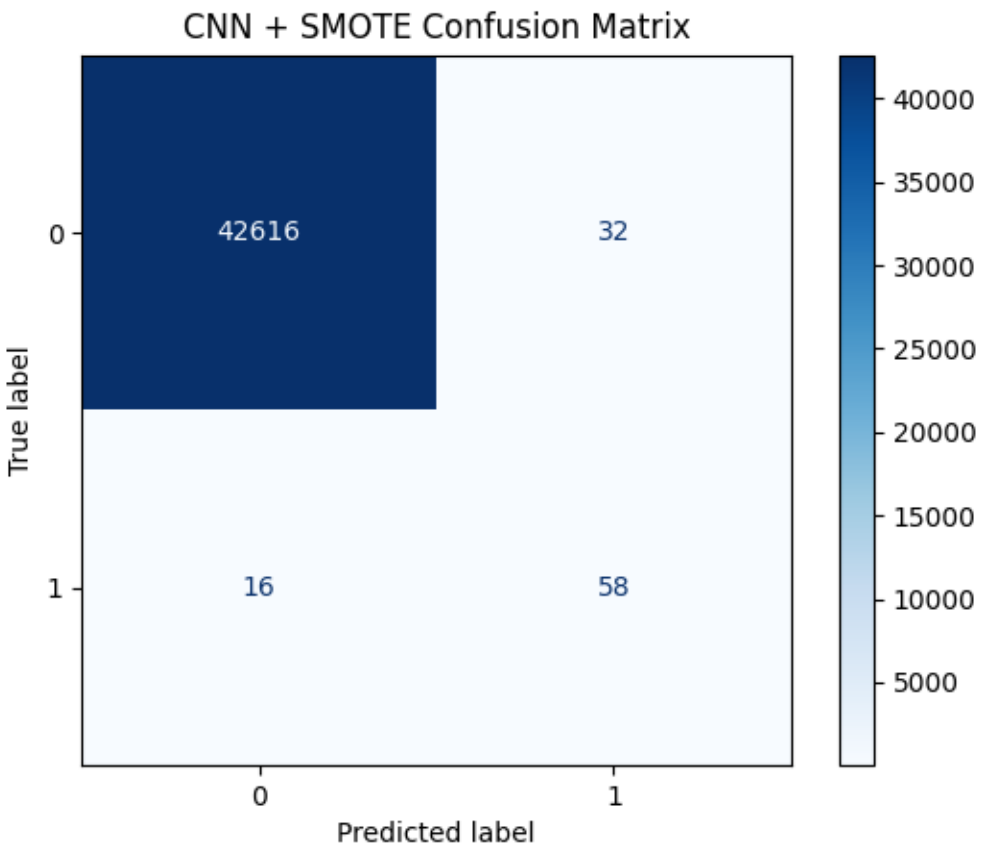


3. CNN_confusionmatrix_withSmote_withlr0.01_andepochs30

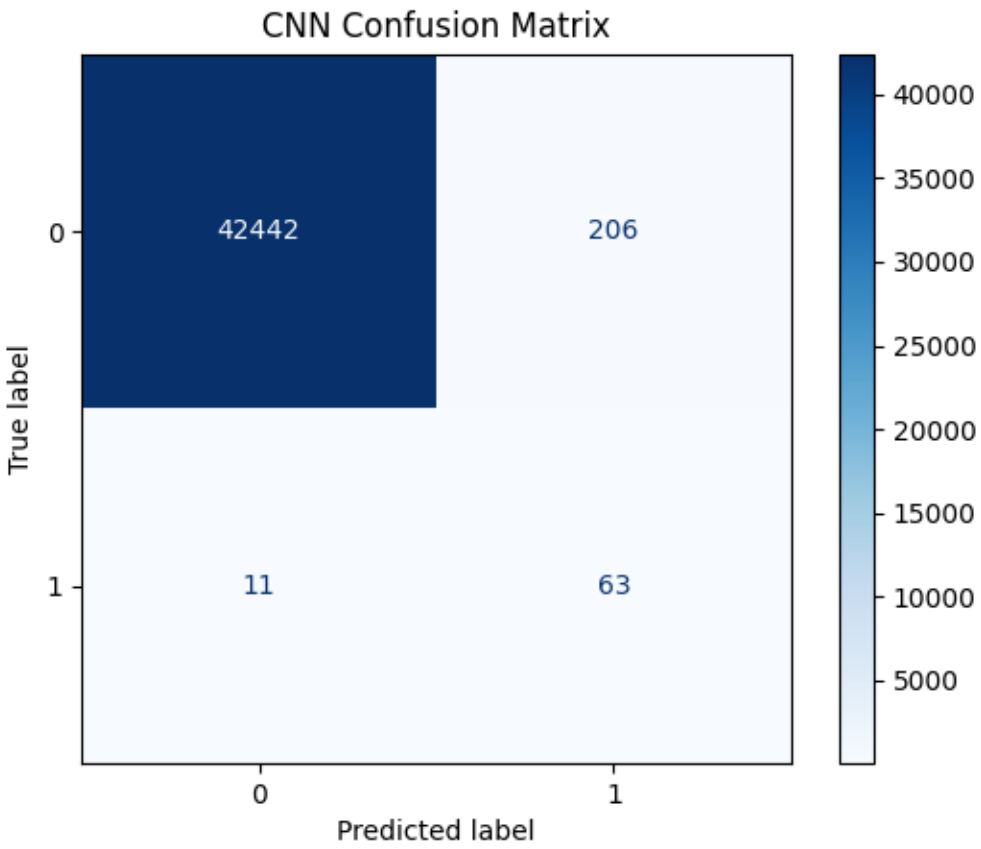
Loss function comparison:

In among all the models, CNN model with lr=0.001 and epoch=30 has minimum error than others. Please see below table.

Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC	Training_Loss_Final	SMOTE
Logistic regression model	**0.9991053**	**0.891304**	**0.554054**	**0.6833333**	**0.977621**		**N/A**
CNN base model with learning_rate=0.01 and epochs=30	0.9988765	0.6444444	0.7837838	0.70731707	0.9402946	0.0032	Applied
CNN model with learning_rate=0.001 and epochs=30	0.9992978	0.8055556	0.7837838	0.79452055	0.9627718	0.001	Applied
CNN model with learning_rate=0.01 and epochs=15	0.9988998642385656	0.6483516483516484	0.7972972972972973	0.7151515151515152	0.9479209442982657	0.00461811	Applied
CNN model with learning_rate=0.01 and epochs=30	0.9947567997752914	0.2262773722627737	0.8378378378378378	0.3563218390804598	0.9612688976258196	0.7469	Not Applied
Foundation model with learning_rate=0.01 and epochs=30	0.9946866	0.2238267	0.83783783	0.35327635	0.9648944	0.72972743	Applied



4.CNN_confusionmatrix_withoutSmote_withlr0.01_andepochs30_dropoutrate0.2



The CNN model with using SMOTE are classifying correctly than the not using SMOTE as the class=1 labels are very less in original data.

11. Team Collaboration • Describe how the team worked together. • Explain what was achieved collectively.

We have been meeting every weekend and discussing the project requirements, model building techniques and code reviews, Presentation preparation. Final we made Video and explained our approaches high level. Overall we learned lot by discussing the CNN model building related concepts and model selecting parameters etc. We feel that we got goos hands on experience on CNN model building.

12. Individual Contributions • Clearly describe each team member's individual contributions.

We both worked separately and built logistic model, CNN_basemodel, compared our models using different techniques and comparing CNN base model Vs Foundation model to compare the results. We both got similar results.

13. Conclusion • Summarize key results and findings. • Discuss lessons learned and possible future improvements.

Summary_Key_results:

Model	**Accuracy**	**Precision**	**Recall**	**F1_Score**	**ROC_AUC**	**Training_Loss_Final**	**SMOTE**
Logistic regression model	**0.9991053**	**0.891304**	**0.554054**	**0.6833333**	**0.977621**		**N/A**
CNN base model with learning_rate=0.01 and epochs=30	0.9988765	0.6444444	0.7837838	0.70731707	0.9402946	0.0032	Applied
CNN model with learning_rate=0.001 and epochs=30	0.9992978	0.8055556	0.7837838	0.79452055	0.9627718	0.001	Applied
CNN model with learning_rate=0.01 and epochs=15	0.9988998642385656	0.6483516483516484	0.7972972972972973	0.7151515151515152	0.9479209442982657	0.00461811	Applied
CNN model with learning_rate=0.01 and epochs=30	0.9947567997752914	0.22627737226277372	0.8378378378378378	0.3563218390804598	0.9612688976258196	0.7469	Not Applied
Foundation model with learning_rate=0.01 and epochs=30	0.9946866	0.2238267	0.83783783	0.35327635	0.9648944	0.72972743	Applied

Discuss lessons learned and possible future improvements: We can hyper tune more parameters- batch size, drop out rate etc and train the models to get better.